

Ascend C 算子开发流程

目 录

1 引言：为什么要自己开发算子	2
2 什么是算子	2
3 两种算子开发模式	3
3.1 快速开发模式	3
3.2 标准开发模式	3
3.3 两种模式对比	3
4 算子原型定义	3
5 msOpGen 工程生成	4
6 Kernel 侧实现	5
6.1 KernelAdd 类与核函数	6
6.2 Init 函数逐行解析	6
7 Host 侧实现	7
7.1 Tiling 实现	8
7.2 Shape 推导	9
7.3 算子原型注册	9
8 编译与部署	10
8.1 CMakePresets.json	10
8.2 编译算子包	11
8.3 部署算子包	11
8.4 源码发布与二进制发布	11
9 PyTorch 适配	11
9.1 前期准备	12
9.2 算子注册分发	12
9.3 适配插件实现	12
10 实战案例：AddCustom 完整流程	13
11 整网算子替换：YOLOV3	13
12 本章你将学会	14
13 要点速查	14
14 小结	15

1 引言：为什么要自己开发算子

一般情况下，开发者无须自己开发算子，深度学习框架已自带丰富的算子库。但当我们第三方框架（如 **PyTorch**（PyTorch）、**TensorFlow**（TensorFlow））的网络迁移到昇腾 AI 处理器时，可能会遇到以下场景：

1. **框架算子不被昇腾支持**：训练场景下，将框架的网络训练脚本迁移到昇腾 AI 处理器时遇到了不支持的算子；推理场景下，使用 ATC 工具将框架模型转换为适配昇腾的离线模型时遇到了不支持的算子。此时需要开发者用 Ascend C 自行编写。
2. **性能不足**：网络调优时，发现某算子性能较低，影响整体网络性能，需要重新开发一个高性能算子替换性能较低的算子。
3. **新研究算子**：学术界提出的新算子尚需数月才能进入框架主分支，研究者需要提前自行实现。
4. **利用 AI 处理器加速数学运算**：推理场景下，若应用程序中的某些逻辑涉及数学运算（如查找最大值、数据类型转换），开发者可将这些操作通过自定义算子的方式实现，让算子在 AI 处理器上运行，达到性能提升的目的。

直觉 不妨把算子开发想成“给昇腾芯片写专属驱动程序”。框架自带的算子是通用驱动，虽然能用但未必针对你的芯片优化过；自己开发算子就是针对昇腾 AI Core 的硬件特性量身定制，既能让不被支持的算子跑起来，又能榨干硬件性能。

Ascend C 提供了从原型定义到 PyTorch 适配的完整开发流程，让自定义算子可以无缝接入现有训练框架。本章将带你走完这条完整路径。

2 什么是算子

算子（Operator）在深度学习网络中对应网络中一个层或节点的计算逻辑。在数学中，算子是函数空间到函数空间上的映射 $O: X \rightarrow X$ ；广义地讲，对任何函数进行某一项操作都可以认为是一个算子，比如微分算子、不定积分算子等。常见算子举例：tanh、ReLU、Conv2D 等。

直觉 你可以把算子理解为一个“加工站”：原材料（输入张量）进去，经过加工（计算逻辑），产出成品（输出张量）。神经网络的每一层都是一个算子，多个算子串联起来就构成了完整的计算图。

从开发视角看，一个完整的算子交付件包含以下组成部分：

组成部分	说明
算子原型定义	JSON 格式，描述输入输出的名称、类型、数据格式
Kernel 侧代码	运行在 AI Core 上的核函数实现
Host 侧代码	Tiling 切分、Shape 推导、原型注册
编译脚本	CMakePresets.json 等，配置 CANN 包路径
算子适配插件	用于 PyTorch 等框架的接口适配

3 两种算子开发模式

Ascend C 提供两种算子开发模式，适用于不同阶段的需求。

3.1 快速开发模式

快速开发模式（Quick Development Mode）适合原型验证和学习阶段。开发者只需完成算子核函数的开发，通过内核调用符方式直接在 Host 端启动核函数运行：

```
constexpr int32_t BLOCK_NUM = 8;
add_custom<<<BLOCK_NUM>>>(xDevice, yDevice, zDevice, workspace, tiling);
```

这种模式的特点：

- 无需编写算子原型 JSON
- 无需编译部署为 .so 库
- 无需适配 ACLNN 接口
- 改完代码即可运行，迭代速度快

快速开发模式的缺点是无法被框架或其他应用调用，仅适合验证算子逻辑正确性。一旦验证通过，仍需转入标准开发模式进行正式交付。

3.2 标准开发模式

标准开发模式（Standard Development Mode）是算子正式交付的完整流程，需完成算子交付件的开发和应用程序的开发。通过这种方式开发的算子可以通过三种途径被调用：

- **ACLNN 接口**：C/C++ 应用直接调用 aclnnXxx 系列 API。
- **ACLOP 接口**：通过算子加载机制动态调用单算子模型。
- **PyTorch Adapter**：在 PyTorch 中像普通 torch 算子一样调用。

3.3 两种模式对比

对比维度	快速开发模式	标准开发模式
开发内容	仅 Kernel 核函数	完整算子交付件 + 应用程序
调用方式	内核调用符	ACLNN / ACLOP / PyTorch Adapter
适用场景	原型验证、学习	正式交付、框架集成
迭代速度	快	较慢（需编译部署）
可被框架调用	否	是

4 算子原型定义

算子原型定义使用 JSON 格式描述算子的接口信息，包括输入输出名称、数据类型、数据格式等。以 AddCustom 算子为例，假设原型定义文件命名为 add_custom.json，存储路径为 \$HOME/sample：

```
[
  {
    "op": "AddCustom",
```

```

    "language": "cpp",
    "input_desc": [
        {
            "name": "x",
            "param_type": "required",
            "format": ["ND"],
            "type": ["fp16"]
        },
        {
            "name": "y",
            "param_type": "required",
            "format": ["ND"],
            "type": ["fp16"]
        }
    ],
    "output_desc": [
        {
            "name": "z",
            "param_type": "required",
            "format": ["ND"],
            "type": ["fp16"]
        }
    ]
}
]

```

关键字段说明：

- op: 算子名称，需全局唯一。
- language: 开发语言，cpp 代表基于 Ascend C 编程框架。
- input_desc / output_desc: 输入输出张量的描述列表，包含参数名称、参数存储格式、参数数据类型。
- param_type: required 表示必选输入，optional 表示可选。
- format: 数据格式，ND 表示任意维度，NCHW 表示四维布局。
- type: 支持的数据类型列表，如 fp16、float。

直觉 原型 JSON 就像算子的“身份证”，它告诉 CANN 运行时：这个算子叫什么名字，接受几个输入，每个输入是什么类型和格式，产出什么输出。有了这张身份证，CANN 才能正确识别和调度你的算子。

5 msOpGen 工程生成

明确原型定义后，使用 CANN 开发套件包提供的 **msOpGen** 工具自动生成完整的算子工程目录，包括 Host 侧代码实现文件、Kernel 侧实现文件、算子适配插件以及工程编译配置文件等。工具路径为 {Ascend_Home_Dir}/python/site-packages/bin/msopgen，其中 Ascend_Home_Dir 在 root 权限下为 /usr/local/Ascend，无 root 权限时为用户安装路径。

生成命令以 AddCustom 为例：

```

${INSTALL_DIR}/python/site-packages/bin/msopgen gen \
  -i $HOME/sample/add_custom.json \
  -c ai_core-<soc_version> \
  -lan cpp \
  -out $HOME/sample/AddCustom

```

各参数含义：

- -i: 指定算子原型定义文件 add_custom.json 所在路径。
- -c: ai_core-<soc_version> 代表算子在 AI Core 上执行，<soc_version> 为昇腾 AI 处理器型号（如 ascend910b）。
- -lan: cpp 代表基于 Ascend C 编程框架，使用 C++ 开发。
- -out: 生成文件所在路径，可配置为绝对路径或相对路径，需具有读写权限。

生成的工程目录结构：

```

AddCustom
├── build.sh
├── cmake
│   ├── config.cmake
│   ├── func.cmake
│   ├── intf.cmake
│   ├── makeself.cmake
│   └── util
├── CMakeLists.txt
├── CMakePresets.json          // 编译配置项
├── framework
├── op_host                   // Host 侧实现文件
│   ├── add_custom_tiling.h  // 算子 tiling 定义文件
│   ├── add_custom.cpp       // 原型注册、shape 推导、tiling 实现
│   └── CMakeLists.txt
├── op_kernel                 // Kernel 侧实现文件
│   ├── CMakeLists.txt
│   └── add_custom.cpp       // 算子代码实现文件
└── scripts

```

op_host 目录交付给主机完成数据切分工作，*op_kernel* 目录实现算子在 AI Core 上的计算逻辑。两者分工明确：*Host* 负责“怎么切”，*Kernel* 负责“怎么算”。

6 Kernel 侧实现

Kernel 侧代码运行在 AI Core 上，是算子的核心计算逻辑。在实际开发场景中，算子的 shape 和数据类型支持动态变化，场景比固定 shape 更加灵活和复杂。**动态 shape 算子**（Dynamic Shape Operator）将形状通过核函数入参传入，参与内部逻辑计算，从而适配不同 shape 的使用场景。

直觉 固定 shape 算子像一条只生产单一规格产品的流水线，动态 shape 算子则像一条可调节的流水线，能根据订单要求（入参 shape）自动调整生产参数。动态 shape 的关键是：Host 侧计算好 tiling 参数后传入 Kernel，Kernel 据此动态分配内存和切分数据。

6.1 KernelAdd 类与核函数

Kernel 侧实现的核心是 KernelAdd 类和 add_custom 核函数。以下是动态 shape 版本的代码框架：

```
constexpr int32_t BUFFER_NUM = 2;    // tensor num for each queue

class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z,
                                uint32_t totalLength, uint32_t tileNum) {
        ...
    }
    __aicore__ inline void Process() {
        ...
    }
private:
    ...
    uint32_t blockLength;
    uint32_t tileNum;
    uint32_t tileLength;
};

extern "C" __global__ __aicore__ void add_custom(
    GM_ADDR x, GM_ADDR y, GM_ADDR z,
    GM_ADDR workspace, GM_ADDR tiling) {
    GET_TILING_DATA(tiling_data, tiling);
    KernelAdd op;
    op.Init(x, y, z, tiling_data.totalLength, tiling_data.tileNum);
    op.Process();
}
```

核函数 add_custom 的三步逻辑：

1. GET_TILING_DATA(tiling_data, tiling)：从 tiling 指针获取 Host 侧传入的 tiling 信息。
2. KernelAdd op;：实例化算子类。
3. op.Init(...) 和 op.Process()：初始化并执行计算。

6.2 Init 函数逐行解析

Init 函数负责根据 tiling 信息和硬件感知获取切分参数，并查找入参地址、分配内存：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z,
                             uint32_t totalLength, uint32_t tileNum) {
    ASSERT(GetBlockNum() != 0 && "block dim can not be zero!");
    this->blockLength = totalLength / GetBlockNum();
    this->tileNum = tileNum;
    ASSERT(tileNum != 0 && "tile num can not be zero!");
    this->tileLength = this->blockLength / tileNum / BUFFER_NUM;

    xGm.SetGlobalBuffer((__gm__ DTYPE_X*)x +
```

```

        this->blockLength * GetBlockIdx(), this->blockLength);
yGm.SetGlobalBuffer((__gm__ DTYPE_Y*)y +
        this->blockLength * GetBlockIdx(), this->blockLength);
zGm.SetGlobalBuffer((__gm__ DTYPE_Z*)z +
        this->blockLength * GetBlockIdx(), this->blockLength);

pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(DTYPE_X));
pipe.InitBuffer(inQueueY, BUFFER_NUM, this->tileLength * sizeof(DTYPE_Y));
pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->tileLength * sizeof(DTYPE_Z));
}

```

逐行解析：

1. `ASSERT(GetBlockNum() != 0 ...)`: 断言 block 数量不为零，防止除零错误。
2. `this->blockLength = totalLength / GetBlockNum()`: 计算每个 block 处理的元素数。
3. `this->tileNum = tileNum`: 保存 tile 切分数目。
4. `this->tileLength = this->blockLength / tileNum / BUFFER_NUM`: 计算每个 tile buffer 的元素数，除以 `BUFFER_NUM`（双缓冲）实现流水线并行。
5. `xGm.SetGlobalBuffer(...)`: 为当前 block 设置 Global Memory 地址，`GetBlockIdx()` 确定当前 block 处理的数据段偏移。
6. `pipe.InitBuffer(...)`: 为输入输出队列分配 Local Memory 缓冲区，`BUFFER_NUM = 2` 实现双缓冲流水线。

例 假设输入张量 `x` 的 shape 为 [4096]（4096 个 fp16 元素），`BLOCK_DIM = 8`，`TILE_NUM = 8`，`BUFFER_NUM = 2`。

Host 侧 `TilingFunc` 计算并传入：`totalLength = 4096`，`tileNum = 8`。

Kernel 侧 `Init` 计算：

- `blockLength = 4096 / 8 = 512`（每个 block 处理 512 个元素）
- `tileLength = 512 / 8 / 2 = 32`（每个 tile buffer 持有 32 个元素）

数据分布：Block 0 处理元素 [0, 511]，Block 1 处理 [512, 1023]，以此类推，Block 7 处理 [3584, 4095]。每个 block 内部运行 8 次 tile 循环，每次搬运 32 个元素到 Local Memory 进行计算，双缓冲让搬运与计算重叠执行。

7 Host 侧实现

Host 侧代码运行在 Host CPU 上，负责三件事：**Tiling 切分**、**Shape 推导**（`InferShape`）、**算子原型注册**。

直觉 如果说 Kernel 侧是“一线工人”负责具体计算，Host 侧就是“车间调度员”：它根据输入数据的形状和硬件资源，计算数据怎么切分（Tiling），推导输出张量的形状（`InferShape`），再把算子的所有信息注册到运行时（原型注册），让 CANN 图引擎能识别和调用这个算子。

7.1 Tiling 实现

Tiling 实现计算数据切分过程相关的参数，比如每次计算的数据量大小。首先在 `add_custom_tiling.h` 中定义和注册 tiling 参数：

```
#include "register/tilingdata_base.h"

namespace optiling {
BEGIN_TILING_DATA_DEF(TilingData)
    TILING_DATA_FIELD_DEF(uint32_t, totalLength);
    TILING_DATA_FIELD_DEF(uint32_t, tileNum);
END_TILING_DATA_DEF;

REGISTER_TILING_DATA_CLASS(AddCustom, TilingData)
}
```

`BEGIN_TILING_DATA_DEF` 和 `END_TILING_DATA_DEF` 之间的字段就是 Host 侧计算后传递给 Kernel 侧的 tiling 参数。这里定义了 `totalLength`（数据全长）和 `tileNum`（切分块数）。

然后在 `add_custom.cpp` 中实现 `TilingFunc`：

```
namespace optiling {
const uint32_t BLOCK_DIM = 8;
const uint32_t TILE_NUM = 8;

static ge::graphStatus TilingFunc(ge::TilingContext* context) {
    TilingData tiling;
    uint32_t totalLength = context->GetInputTensor(0)->GetShapeSize();
    context->SetBlockDim(BLOCK_DIM);

    tiling.set_totalLength(totalLength);
    tiling.set_tileNum(TILE_NUM);

    tiling.SaveToBuffer(context->GetRawTilingData()->GetData(),
                       context->GetRawTilingData()->GetCapacity());
    context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
    context->SetTilingKey(1);

    size_t *currentWorkspace = context->GetWorkspaceSizes(1);
    currentWorkspace[0] = 0;

    return ge::GRAPH_SUCCESS;
}
}
```

逐行解析：

1. `const uint32_t BLOCK_DIM = 8`：定义使用的 block 数量为 8。
2. `context->GetInputTensor(0)->GetShapeSize()`：从运行时上下文获取第一个输入张量的元素总数。
3. `context->SetBlockDim(BLOCK_DIM)`：设置 block 维度，告诉运行时启动 8 个 block。
4. `tiling.set_totalLength(totalLength)` 和 `tiling.set_tileNum(TILE_NUM)`：填充 tiling 数据。
5. `tiling.SaveToBuffer(...)`：将 tiling 数据序列化到运行时提供的缓冲区。

6. context->SetTilingKey(1): 设置 tiling key, 用于 Kernel 侧区分不同的 tiling 策略。
7. currentWorkspace[0] = 0: 设置 workspace 大小为 0 (AddCustom 不需要额外 workspace)。

7.2 Shape 推导

InferShape (形状推导) 根据算子的输入张量描述、算子逻辑及算子属性, 推理出输出张量的描述, 包括张量的 Shape、数据类型及数据排布格式。这样算子构图准备阶段就可以为所有张量静态分配内存, 避免动态内存分配带来的开销。对于 AddCustom, 输出 z 的 shape 与输入 x 相同:

```
namespace ge {
static ge::graphStatus InferShape(ge::InferShapeContext* context) {
    const ge::Shape* x1_shape = context->GetInputShape(0);
    ge::Shape* y_shape = context->GetOutputShape(0);
    *y_shape = *x1_shape;
    return GRAPH_SUCCESS;
}
}
```

这部分代码由工具自动生成, 开发者通常无须修改。对于 AddCustom 这种逐元素运算, 输出 shape 等于输入 shape 是最常见的情况。

7.3 算子原型注册

算子原型注册将算子名称、输入输出信息、Tiling 函数、InferShape 函数等绑定到 CANN 运行时。注册代码在 add_custom.cpp 中:

```
namespace ops {
class AddCustom : public OpDef {
public:
    explicit AddCustom(const char* name) : OpDef(name) {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT})
            .Format({ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND});
        this->Input("y")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT})
            .Format({ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND});
        this->Output("z")
            .ParamType(REQUIRED)
            .DataType({ge::DT_FLOAT})
            .Format({ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND});

        this->SetInferShape(ge::InferShape);
        this->AICore()
            .SetTiling(optiling::TilingFunc);
    }
};
}
```

```

        this->AICore().AddConfig("ascend910");
        this->AICore().AddConfig("ascend310p");
    }
};
OP_ADD(AddCustom);
}

```

逐行解析：

1. `this->Input("x").ParamType(REQUIRED)`：定义输入 `x` 为必选参数。
2. `.DataType({ge::DT_FLOAT})`：设置数据类型为 `float`。
3. `.Format({ge::FORMAT_ND})`：设置数据格式为 `ND`（任意维度）。
4. `.UnknownShapeFormat({ge::FORMAT_ND})`：设置动态 `shape` 时的格式。
5. `this->SetInferShape(ge::InferShape)`：关联 `Shape` 推导函数。
6. `this->AICore().SetTiling(optiling::TilingFunc)`：关联 `Tiling` 实现函数。
7. `this->AICore().AddConfig("ascend910")`：注册算子支持的 AI 处理器型号。
8. `OP_ADD(AddCustom)`：将算子注册到 CANN 运行时。

注册后，`AddCustom` 算子就可以被 CANN 图引擎识别和调用。

8 编译与部署

完成 Kernel 侧和 Host 侧代码编写后，需要编译算子包并部署到 CANN 环境中。

8.1 CMakePresets.json

`CMakePresets.json` 是编译配置项文件，最重要的配置是 `ASCEND_CANN_PACKAGE_PATH`，指向本地 CANN 软件包安装路径：

```

{
  "ASCEND_CANN_PACKAGE_PATH": {
    "type": "PATH",
    "value": "/usr/local/Ascend/latest"
  },
  "ENABLE_CROSS_COMPILE": {
    "type": "BOOL",
    "value": "False"
  },
  "CMAKE_CROSS_PLATFORM_COMPILER": {
    "type": "PATH",
    "value": "/usr/bin/aarch64-linux-gnu-g++"
  },
  "vendor_name": {
    "type": "STRING",
    "value": "customize"
  }
}

```

关键字段：

- `ASCEND_CANN_PACKAGE_PATH`：CANN 软件包安装路径，编译时据此找到头文件和库文件。
- `ENABLE_CROSS_COMPILE`：是否使能交叉编译，请根据实际环境配置。
- `CMAKE_CROSS_PLATFORM_COMPILER`：交叉编译工具路径。

- `vendor_name`: 厂商名称, 默认 `customize`, 其取值会影响部署算子包时对应的部署目录。

8.2 编译算子包

完成所有代码修改后, 在工程文件夹下执行编译脚本:

```
./build.sh
```

编译时会在算子工程根目录下生成 `build_out` 目录, 编译完成后生成的算子 `run` 包存放在 `build_out` 目录下。

8.3 部署算子包

部署有两种方式:

1. **默认部署 (推荐)**: 直接执行 `./custom_opp_xxx.run`, 算子会部署在 `<ASCEND_CANN_PACKAGE_PATH>/opp/vendors` 目录下。
2. **指定部署目录**: 执行 `./custom_opp_xxx.run --install-path=xxx`, 部署到指定目录。此时需要将自定义安装目录添加到 `ASCEND_CUSTOM_OPP_PATH` 环境变量, 才能使算子在当前环境生效。

部署后的目录结构:

```

opp
├── vendors                                // 自定义算子所在目录
│   ├── config.ini
│   └── customize                          // 厂商名称, 默认为 customize
│       ├── framework                    // 自定义算子插件库
│       ├── op_api
│       │   ├── include
│       │   │   └── aclnn_xx.h          // 算子调用 API 声明文件
│       │   └── lib
│       │       └── libcust_opapi.so
│       ├── op_impl
│       │   ├── ai_core
│       │   └── tbe
│       └── op_proto                      // 自定义算子原型库所在目录

```

8.4 源码发布与二进制发布

编译 Kernel 侧代码的方式分为两种:

- **源码发布**: 不对 Kernel 侧实现进行编译, 保留相关文件, 支持算子在线编译或通过 ATC 模型转换方式编译算子。兼容性好, 但需要编译环境。
- **二进制发布**: 生成描述算子相关信息的 JSON 文件和二进制文件, 支持单算子 API 执行、PyTorch 框架下的单算子调用、动态网络中的算子调用场景。使用方便, 但与特定 CANN 版本和芯片型号绑定。

9 PyTorch 适配

为了让自定义算子在 PyTorch 中直接使用, 需要进行 PyTorch 适配。适配流程主要包括两个步骤: **算子注册分发** 和 **适配插件实现**。

9.1 前期准备

在进行 PyTorch 适配前，需要：

1. 完成自定义算子的编译部署。
2. 在编译部署时，将 CMakePresets.json 中的 ENABLE_BINARY_PACKAGE 设置为 True，将算子的二进制部署到当前环境。
3. 编译部署后，将算子接口库的路径设置到共享库的查找路径下。

9.2 算子注册分发

对于自定义算子，由于没有具体的算子定义，需要在 npu_native_functions.yaml 文件中给出定义，以便对算子进行结构化解析，实现自动化注册和 Python 接口绑定：

```
backend: NPU
cpp_namespace: at_npu::native
supported:
  - add.Tensor
  - add.Scalar
autograd:
  - maxpool2d
custom:
  - func: npu_add_custom(Tensor x, Tensor y) -> Tensor
custom_autograd:
  - func: npu_convolution(Tensor input, Tensor weight, Tensor? bias, ...) -> Tensor
```

各字段含义：

- backend 和 cpp_namespace：声明插件中开发算子的命名空间。
- supported：声明已支持的与 PyTorch 原生函数对齐的算子。
- autograd：声明已支持的具有前反向操作的算子。
- custom：声明自定义算子，这里注册了 npu_add_custom。
- custom_autograd：声明自定义继承自原生函数的自定义算子。

9.3 适配插件实现

适配插件文件的命名格式一般为 <算子名称>+<KernelNpu>.cpp，如 AddCustomKernelNpu.cpp。插件实现 PyTorch 原生算子的输入参数、输出参数和属性的格式转换，使转换后的格式与自定义算子的格式相同：

```
#include <torch/csrc/autograd/custom_function.h>
#include "torch_npu/csrc/framework/utils/OpAdapter.h"
#include "torch_npu/csrc/aten/NPUNativeFunctions.h"
#include "torch_npu/csrc/aten/ops/op_api/op_api_common.h"

namespace at_npu {
namespace native {
using torch::autograd::Function;
using torch::autograd::AutogradContext;

at::Tensor NPUNativeFunctions::npu_add_custom(
    const at::Tensor& x, const at::Tensor& y) {
```

```

at::Tensor result = OpPreparation::ApplyTensor(x);
EXEC_NPU_CMD(aclnnAddCustom, x, y, result);
return result;
}
}
}

```

逐行解析：

1. `OpPreparation::ApplyTensor(x)`：根据输入 `x` 的形状和类型预分配输出张量。
2. `EXEC_NPU_CMD(aclnnAddCustom, x, y, result)`：调用 ACLNN 接口执行自定义算子，将 PyTorch 张量传递给 `aclnnAddCustom`。

编译安装插件后，在 PyTorch 中即可直接调用：

```

import torch_npu
z = torch_npu.npu_add_custom(x, y)

```

10 实战案例：AddCustom 完整流程

以下是将上述各步骤串联的完整 AddCustom 开发流程：

1. **编写原型 JSON**：在 `$HOME/sample/add_custom.json` 中定义 `x`、`y` 输入和 `z` 输出。
2. **msOpGen 生成工程**：执行 `msopgen gen -i add_custom.json -c ai_core-
<soc_version> -lan cpp -out ./AddCustom`。
3. **编写 Kernel 侧**：在 `op_kernel/add_custom.cpp` 中实现 `KernelAdd` 类和 `add_custom` 核函数，处理动态 shape。
4. **编写 Host 侧**：在 `op_host/add_custom_tiling.h` 中定义 `tiling` 参数，在 `op_host/add_custom.cpp` 中实现 `TilingFunc`、`InferShape`、算子原型注册。
5. **配置编译**：修改 `CMakePresets.json` 中的 `ASCEND_CANN_PACKAGE_PATH`。
6. **编译部署**：执行 `./build.sh`，然后执行 `./build_out/custom_opp_xxx.run`。
7. **验证调用**：通过 ACLNN 接口或 PyTorch 适配调用。

例 以 shape 为 [4096] 的 fp16 加法为例，完整数据流如下：

Host 侧：

- `TilingFunc` 读取 `totalLength = 4096`，设置 `BLOCK_DIM = 8`，`tileNum = 8`
- 序列化 `TilingData` 并传入 Kernel

Kernel 侧（以 Block 0 为例）：

- `blockLength = 4096 / 8 = 512`
- `tileLength = 512 / 8 / 2 = 32`
- `xGm.SetGlobalBuffer(x + 0, 512)`：Block 0 处理元素 [0, 511]
- 循环 8 次 `tile`，每次搬运 32 个 fp16 元素到 Local Memory
- 双缓冲让数据搬运与计算重叠，最大化 AI Core 利用率

11 整网算子替换：YOLOV3

以适配昇腾芯片的 **PyTorch-YOLOV3** 网络为例，将网络中的加法算子 + 替换成使用 Ascend C 实现的自定义加法算子 `AddCustom`。

YOLOV3 网络的 backbone 模块使用 Darknet 结构，其中的 ResBlock 块用到了 + 算子，文件目录为 ./mmdet/models/backbones/darknet.py。

替换方法：使用 torch_npu.npu_add_custom(x, y) 接口调用自定义 add_custom 算子替换原有的 add 算子：

```
# 原始 darknet.py 中的加法
# out = x + y
```

```
# 替换为昇腾自定义算子
```

```
import torch_npu
out = torch_npu.npu_add_custom(x, y)
```

替换后，YOLOV3 前向计算中所有逐元素加法都在昇腾 AI Core 上并行执行，相比 CPU 实现可获得显著加速。这种适配方式无需修改模型结构，只需替换关键算子调用，是昇腾平台模型迁移的常用策略。

YOLOV3 网络获取网址：https://gitee.com/ascend/ModelZoo-PyTorch/tree/master/PyTorch/built-in/cv/detection/YoloV3_ID1790_for_PyTorch

12 本章你将学会

1. 理解算子开发的需求场景，判断何时需要自行开发算子。
2. 区分快速开发模式与标准开发模式，选择适合的开发路径。
3. 使用 msOpGen 工具基于原型 JSON 生成完整的算子工程目录。
4. 编写 Kernel 侧动态 shape 核函数和 Host 侧 Tiling、InferShape、原型注册代码。
5. 完成算子编译部署并通过 PyTorch 适配在整网中替换算子。

13 要点速查

步骤	关键文件 / 工具	核心内容
原型定义	add_custom.json	JSON 描述输入输出名称、类型、格式
工程生成	msopgen	自动生成 op_host、op_kernel 目录结构
Kernel 侧	op_kernel/add_custom.cpp	KernelAdd 类、核函数、动态 shape
Tiling 定义	op_host/add_custom_tiling.h	BEGIN_TILING_DATA_DEF 定义 tiling 字段
Tiling 实现	op_host/add_custom.cpp	TilingFunc 计算切分参数
Shape 推导	op_host/add_custom.cpp	InferShape 推导输出 shape
原型注册	op_host/add_custom.cpp	OpDef 子类、OP_ADD 宏
编译配置	CMakePresets.json	ASCEND_CANN_PACKAGE_PATH 路径
编译	./build.sh	生成 build_out/custom_opp_xxx.run
部署	./custom_opp_xxx.run	部署到 opp/vendors 目录
PyTorch 注册	npu_native_functions.yaml	custom 段声明 npu_add_custom
PyTorch 插件	AddCustomKernelNpu.cpp	EXEC_NPU_CMD 调用 ACLNN

14 小结

本章梳理了 Ascend C 算子开发的完整流程。从判断是否需要自行开发算子开始，到选择快速开发模式或标准开发模式，再到通过 msOpGen 生成工程骨架、编写 Kernel 侧和 Host 侧代码、配置 CMakePresets.json 编译部署，最终通过 PyTorch 适配让自定义算子无缝融入训练框架，并在 YOLOV3 整网中完成算子替换。这一流程将硬件层面的算子开发与上层框架对接起来，是算子从实验室走向生产环境的关键路径。

讲义基于 Ascend C 算子开发流程课程内容编写